

Breaze & Harold Veterinary System

Diseño y documentación API First

Microservicio bh-core



Programación Backend

Jhon Harold Morales Velez

Simón Gaviria Orrego

Universidad Autónoma de Manizales

Ingeniería de Sistemas

25 de Mayo , 2026

Objetivo

El presente documento tiene como objetivo evidenciar el diseño y documentación del contrato OpenAPI del microservicio bh-core, siguiendo el enfoque API First.

Antes de iniciar la implementación, se definieron los endpoints principales del sistema, incluyendo métodos HTTP, rutas, parámetros de entrada, cuerpos de solicitud, respuestas exitosas, códigos de error, esquemas reutilizables y mecanismos de seguridad.

El contrato fue construido en formato YAML para ser validado en Swagger Editor y posteriormente incluido en el repositorio del proyecto.

Este archivo representa el contrato formal de la API y fue diseñado antes de iniciar la implementación del código fuente, cumpliendo con el enfoque API First solicitado para el proyecto.

Alcance del MicroServicio bh-core

El microservicio bh-core concentra la operación principal de la clínica veterinaria Breaze & Harold Veterinary System.

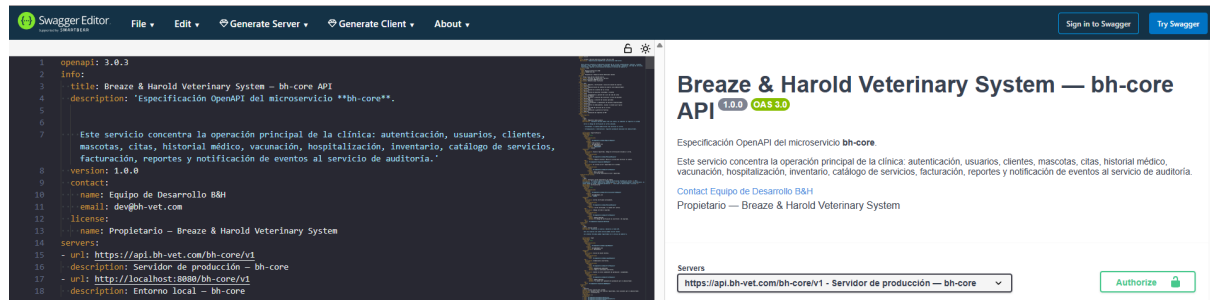
Este servicio cubre los siguientes módulos funcionales:

- Autenticación y registro de usuarios.
- Administración de usuarios y control de cuentas.
- Gestión de clientes.
- Gestión de mascotas.
- Agenda y ciclo de vida de citas.
- Historial médico.
- Vacunación.
- Hospitalización.
- Inventario de medicamentos, vacunas e insumos.
- Catálogo de servicios.
- Facturación.
- Generación de reportes en PDF.
- Notificación de eventos relevantes hacia el servicio bh-audit.

El servicio utiliza autenticación mediante JWT para proteger los endpoints que requieren sesión activa.

Evidencia de validación en Swagger Editor

- Validación del archivo bh-core.yaml en Swagger Editor



La figura anterior muestra la carga del archivo bh-core.openapi.yaml en Swagger Editor, permitiendo visualizar los módulos, endpoints, esquemas y respuestas definidos para el microservicio bh-core.

Módulos documentados en bh-core

Servers		https://api.bh-vet.com/bh-core/v1 - Servidor de producción — bh-core	Authorize 
Auth	Registro, verificación e inicio de sesión de usuarios		▼
Usuarios	Administración de cuentas de usuario (solo administrador)		▼
Clientes	Gestión de clientes de la clínica		▼
Mascotas	Gestión de mascotas vinculadas a clientes		▼
Citas	Agendamiento y gestión del ciclo de vida de citas		▼
Historial Médico	Registro y consulta del historial clínico de mascotas		▼
Vacunación	Registro y consulta de vacunas aplicadas		▼
Hospitalización	Internamiento y seguimiento de mascotas hospitalizadas		▼
Inventario	Control de medicamentos, vacunas e insumos quirúrgicos		▼
Servicios	Catálogo de servicios de la clínica		▼
Facturación	Generación y gestión de facturas		▼
Reportes	Generación de reportes en PDF		▼

Tabla resumen de endpoints principales

- Auth

Auth

Registro, verificación e inicio de sesión de usuarios

POST

/auth/registro

Registrar nuevo usuario

POST

/auth/verificar-correo

Verificar correo electrónico con código

POST

/auth/login

Iniciar sesión

- Usuarios

Usuarios Administración de cuentas de usuario (solo administrador)



GET	/usuarios	Listar usuarios del sistema		
GET	/usuarios/{usuarioId}	Obtener usuario por ID		
PATCH	/usuarios/{usuarioId}/aprobar	Aprobar cuenta de recepcionista o veterinario		
PATCH	/usuarios/{usuarioId}/rechazar	Rechazar cuenta de recepcionista o veterinario		
PATCH	/usuarios/{usuarioId}/suspender	Suspender cuenta de usuario		

- Clientes

Clientes Gestión de clientes de la clínica



GET	/clientes	Listar clientes		
POST	/clientes	Registrar cliente		
GET	/clientes/{clienteId}	Obtener cliente por ID		
PUT	/clientes/{clienteId}	Actualizar datos del cliente		

- Mascotas

Mascotas Gestión de mascotas vinculadas a clientes



GET	/clientes/{clienteId}/mascotas	Listar mascotas de un cliente		
POST	/clientes/{clienteId}/mascotas	Registrar mascota para un cliente		
GET	/mascotas/{mascotaId}	Obtener mascota por ID		
PUT	/mascotas/{mascotaId}	Actualizar datos de la mascota		

- Citas

Citas Agendamiento y gestión del ciclo de vida de citas



GET	/citas	Listar citas		
POST	/citas	Agendar nueva cita		
GET	/citas/{citaId}	Obtener cita por ID		
PATCH	/citas/{citaId}/finalizar	Marcar cita como finalizada		
PATCH	/citas/{citaId}/cancelar	Cancelar cita		
GET	/citas/disponibilidad	Consultar disponibilidad de veterinario		

- Historial Médico

Historial Médico Registro y consulta del historial clínico de mascotas



GET	/mascotas/{mascotaId}/historial	Consultar historial médico de una mascota		
POST	/mascotas/{mascotaId}/historial	Registrar consulta en el historial médico		
GET	/historial/{registroId}	Obtener registro médico por ID		
PUT	/historial/{registroId}	Editar registro médico		

- Vacunación

Vacunación Registro y consulta de vacunas aplicadas



GET	/mascotas/{mascotaId}/vacunas	Consultar vacunas de una mascota		
POST	/mascotas/{mascotaId}/vacunas	Registrar vacuna aplicada		
GET	/vacunas/proximas-a-vencer	Listar mascotas con vacunas próximas a vencer		

- Hospitalización

Hospitalización Internamiento y seguimiento de mascotas hospitalizadas



POST	/mascotas/{mascotaId}/hospitalizacion	Internar mascota		
POST	/hospitalizacion/{hospitalizacionId}/evolucion	Registrar nota de evolución diaria		
PATCH	/hospitalizacion/{hospitalizacionId}/alta	Dar de alta a la mascota		

- Inventario

Inventario Control de medicamentos, vacunas e insumos quirúrgicos



GET	/inventario	Listar productos del inventario		
POST	/inventario	Registrar producto en inventario		
GET	/inventario/{productoId}	Obtener producto por ID		
PUT	/inventario/{productoId}	Actualizar producto del inventario		

- Servicios

Servicios Catálogo de servicios de la clínica



GET	/servicios	Listar servicios del catálogo		
POST	/servicios	Crear nuevo servicio		
GET	/servicios/{servicioId}	Obtener servicio por ID		
PUT	/servicios/{servicioId}	Editar servicio del catálogo		
PATCH	/servicios/{servicioId}/desactivar	Desactivar servicio del catálogo		

- Facturación

Facturación Generación y gestión de facturas



GET	/facturas	Listar facturas		▼
POST	/facturas	Generar factura		▼
GET	/facturas/{facturaId}	Obtener factura por ID		▼
PATCH	/facturas/{facturaId}/anular	Anular factura		▼
GET	/facturas/{facturaId}/pdf	Descargar factura en PDF		▼

- Reportes

Reportes Generación de reportes en PDF



GET	/reportes/citas	Reporte de citas por período (PDF)		▼
GET	/reportes/facturacion	Reporte de facturación por período (PDF)		▼
GET	/reportes/inventario	Reporte de inventario actual (PDF)		▼
GET	/reportes/historial-acciones	Reporte de historial de acciones (PDF)		▼

Descripción de módulos por endpoints

- Auth

/auth/registro : Registrar nuevo usuario

Descripción:

Cualquier persona puede crear una cuenta. Al completar el registro el sistema envía un código de verificación al correo indicado.

cliente: la cuenta queda activa tras verificar el correo.

recepcionista / veterinario: requieren aprobación adicional del administrador.

Ejemplo de request:

```
{
  "nombre": "María García",
  "email": "maria@ejemplo.com",
  "password": "P@ssw0rd123",
  "telefono": "3001234567",
  "rol": "cliente"
}
```


Responses:

201: Usuario registrado. Código de verificación enviado al correo

400: Datos de entrada inválidos o incompletos

409: El correo ya está registrado en el sistema

/auth/verificar-correo: Verificar correo electrónico con código

Descripción:

El usuario ingresa el código de verificación recibido por correo.

Para **clientes**, la cuenta queda activa inmediatamente.

Para **recepcionistas y veterinarios**, la cuenta queda en estado **pendiente_aprobacion** hasta que el administrador lo active

Ejemplo de request:

```
{  
  "email": "maria@ejemplo.com",  
  "código": "482913"  
}
```

Responses:

200: Correo verificado exitosamente.

400: Código inválido o expirado.

404: El recurso solicitado no existe.

/auth/login : Iniciar sesión

Descripción:

Autentica al usuario y devuelve un token JWT. Solo los usuarios con cuenta activa pueden iniciar sesión. Los intentos fallidos quedan registrados en el servicio de auditoría.

Ejemplo de request:

```
{  
  "email": "maria@ejemplo.com",  
  "password": "P@ssw0rd123"  
}
```

Responses:

200: Inicio de sesión exitoso

400: Datos de entrada inválidos o incompletos

401: Credenciales incorrectas

403: Cuenta no activa (pendiente de aprobación o suspendida)

- **Usuarios**

/usuarios : Listar usuarios del sistema

Descripción:

Devuelve todos los usuarios registrados. Solo accesible por el administrador.

Ejemplo de request:

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "nombre": "string",
  "email": "user@example.com",
  "telefono": "string",
  "rol": "cliente",
  "estadoCuenta": "pendiente_verificacion",
  "creadoEn": "2026-05-16T16:46:31.491Z",
  "actualizadoEn": "2026-05-16T16:46:31.491Z"
}
```

Responses:

200: Lista de usuarios.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/usuarios/{usuariold} : Obtener usuario por ID

Descripción: devuelve usuarios por ID

Parámetros: usuariold

Responses:

200: Datos del usuario.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/usuarios/{usuariold}/aprobar: Aprobar cuenta de recepcionista o veterinario

Descripción:

El administrador aprueba manualmente la cuenta de un recepcionista o veterinario que ha verificado su correo y está en estado

pendiente_aprobacion..

Parámetros: usuariold

Responses:

200: Cuenta aprobada.

400: El usuario no está en estado pendiente de aprobación

401: Token de autenticación ausente o inválido

403: El rol usuario no tiene permiso para ejecutar esta acción

404: El recurso solicitado no existe

/usuarios/{usuariold}/rechazar : Aprobar cuenta de recepcionista o veterinario

Descripción: El administrador rechaza manualmente la cuenta de un recepcionista o veterinario.

Ejemplo de request:

```
{  
  "motivo": "El cliente solicitó cancelación por cambio de planes."  
}
```

Responses:

200: Cuenta rechazada.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/usuarios/{usuariold}/suspender: Suspender cuenta de usuario

Descripción:

El administrador desactiva una cuenta activa.

Parámetros: usuariold

Responses:

200: Cuenta suspendida.

400: Datos de entrada inválido o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol usuario no tiene permiso para ejecutar esta acción

404: El recurso solicitado no existe

- Clientes

/clientes: Listar clientes

Descripción: Roles permitidos: recepcionista, administrador.

Parámetros:

busqueda string (query): Filtrar por nombre, apellido o correo

page integer (query): Número de página (base 0)

size integer(query): Elementos por página

Responses:

200: Lista paginada de clientes.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción

/clientes: Registrar cliente

Descripción:

La recepcionista registra un nuevo cliente con sus datos de contacto.

Roles permitidos: Recepcionista

Ejemplo de request:

```
{  
  "nombre": "Juan",  
  "apellido": "Pérez",  
  "email": "user@example.com",  
  "telefono": "string",  
  "documento": "string",  
  "direccion": "string"  
}
```

Responses:

201: Cliente registrado.

400: Datos de entrada inválido o incompletos..

401: Token de autenticación ausente o inválido

403: El rol usuario no tiene permiso para ejecutar esta acción

409: Ya existe un cliente con ese correo o documento.

/clientes/{clientId}: Obtener cliente por ID

Descripción:

El propio cliente puede consultar su información usando su token.

Roles permitidos: Recepcionista y administrador.

Parámetros: clientId

Responses:

200: Datos del cliente

401: Token de autenticación ausente o inválido

403: El rol usuario no tiene permiso para ejecutar esta acción

404: El recurso solicitado no existe.

/clientes/{clientId}: Actualizar datos del cliente

Roles permitidos: Recepcionista

Parámetros: clientId

Ejemplo de Request:

```
{  
  "nombre": "Juan",  
  "apellido": "Pérez",  
  "email": "user@example.com",  
}
```

```
"telefono": "string",  
"documento": "string",  
"direccion": "string"  
}
```

Responses:

200: Cliente actualizado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido

403: El rol del usuario no tiene permiso para ejecutar esta acción

404: El recurso solicitado no existe.

- **Mascotas**

/clientes/{clienteld}/mascotas: Listar mascotas de un cliente

Descripción:Lista todas las mascotas asociadas a un cliente específico.

Roles permitidos: recepcionista, veterinario, administrador y el propio cliente.

Parámetros:

clienteld string(\$uuid) (path): Identificador único del cliente.

Responses:

200: Lista de mascotas del cliente.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/clientes/{clienteld}/mascotas: Registrar mascota para un cliente

Descripción:Roles permitidos: recepcionista.

Parámetros:

clienteld string(\$uuid) (path): Identificador único del cliente.

Ejemplo de request:

```
{  
  "nombre": "Luna",  
  "especie": "Perro",  
  "raza": "Golden Retriever",  
  "color": "Dorado",  
  "fechaNacimiento": "2020-03-15",  
}
```

```
"peso": 12.5
}
```

Responses:

201: Mascota registrada.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/mascotas/{mascotaid}: Obtener mascota por ID**Descripción:**

Permite consultar la información detallada de una mascota registrada en el sistema mediante su identificador único.

Roles permitidos: Recepcionista, Administrador y Veterinario.

Parámetros: mascotaid

Responses:

200: Datos de la mascota.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/mascotas/{mascotaid}: Actualizar datos de la mascota

Descripción: Permite actualizar los datos básicos de una mascota registrada, como nombre, especie, raza, color, fecha de nacimiento y peso.

El peso también puede actualizarse automáticamente cuando se registra una consulta en el historial médico.

Roles permitidos: Recepcionista.

Parámetros: mascotaid:

Ejemplo de response:

```
{
  "nombre": "Luna",
  "especie": "Perro",
  "raza": "Golden Retriever",
  "color": "Dorado",
  "fechaNacimiento": "2020-03-15",
  "peso": 12.5
}
```

Responses:

200: Mascota actualizada correctamente.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- Citas

/citas: Listar citas

Descripción:

cliente: ve sólo sus propias citas.

veterinario: ve solo las citas asignadas a él.

repcionista / administrador: ven todas las citas.

Parámetros:

veterinariold string(\$uuid) (query): Filtrar por veterinario

mascotald string(\$uuid) (query): Filtrar por mascota

estado string (query): Filtrar por estado de la cita. Valores permitidos: confirmada, finalizada, cancelada

fechaDesde string(\$date) (query): Fecha de inicio del rango (YYYY-MM-DD)

fechaHasta string(\$date) (query): Fecha de fin del rango (YYYY-MM-DD)

page integer (query): Número de página (base 0)

size integer (query): Elementos por página

Responses:

200: Lista paginada de citas.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/citas: Agendar nueva cita

Descripción: Agenda una cita y registra el pago de los servicios seleccionados.

El pago es obligatorio; sin él la cita no se confirma.

El sistema valida que no exista cruce de horario para el veterinario asignado.

Al confirmar, se envía un correo automático al cliente.

Roles permitidos: recepcionista, cliente.

Ejemplo de request:

```
{
  "mascotald": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "veterinariold": "7cb94e91-1234-4321-b3fc-9d853f11bcd2",
```

```
"fecha": "2025-08-15",
"hora": "10:00",
"servicioids": [
  "aab5c123-0000-0000-0000-000000000001",
  "aab5c123-0000-0000-0000-000000000002"
],
"pago": {
  "metodo": "tarjeta",
  "referencia": "TXN-20250815-001"
}
}
```

Responses:

201: Cita confirmada y pago registrado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

409: El veterinario ya tiene una cita en ese horario.

422: Pago rechazado o información de pago incompleta.

/citas/{citald}: Obtener cita por ID

Descripción: Permite consultar la información detallada de una cita específica.

Parámetros:

citald string(\$uuid) (path): Identificador único de la cita.

Responses:

200: Datos de la cita.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/citas/{citald}/finalizar: Marcar cita como finalizada

Descripción: Permite marcar una cita como finalizada después de que el veterinario haya registrado el historial médico correspondiente.

El registro clínico de la atención se crea previamente en el endpoint POST **/mascotas/{mascotald}/historial**, donde se documentan el motivo de visita, diagnóstico, tratamiento, medicamentos prescritos, peso actual y próxima visita.

Roles permitidos: Veterinario.

Parámetros: citald

Responses:

200: Cita finalizada correctamente.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/citas/{citald}/cancelar: Cancelar cita

Descripción:

Permite cancelar una cita que aún no ha sido atendida. Para realizar la cancelación, se debe registrar el motivo correspondiente. El reembolso queda a criterio administrativo y no es gestionado por el sistema.

Roles permitidos: Recepcionista y Administrador.

Parámetros: citald

Ejemplo request:

```
{
  "motivo": "El cliente solicitó cancelación por cambio de planes."
}
```

Responses:

200: Cita cancelada correctamente.

400: La cita ya fue atendida o no puede cancelarse.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/citas/disponibilidad: Consultar disponibilidad de veterinario

Descripción:

Permite consultar los horarios disponibles de un veterinario en una fecha específica.

Roles permitidos:Recepcionista y Cliente.

Parámetros: veterinariold y fecha: Fecha a consultar en formato YYYY-MM-DD.

Responses:

200: Horarios disponibles.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- Historial Médico

/mascotas/{mascotaid}/historial: Consultar historial médico de una mascota

Descripción:

Roles permitidos: veterinario, recepcionista, administrador, y el cliente dueño de la mascota.

Parámetros:

mascotaid string(\$uuid) (path): Identificador único de la mascota.

page integer (query): Número de página (base 0).

size integer (query): Elementos por página.

Responses:

200: Historial médico de la mascota.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/mascotas/{mascotaid}/historial: Registrar consulta en el historial médico

Descripción:El veterinario registra el resultado de la consulta al finalizar la cita.

El peso de la mascota se actualiza automáticamente con el valor registrado aquí.

Roles permitidos: veterinario.

Parámetros:

mascotaid string(\$uuid) (path): Identificador único de la mascota.

Ejemplo de request:

```
{
  "citald": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "motivoVisita": "string",
  "diagnostico": "string",
  "tratamiento": "string",
  "medicamentosPrescritos": [
    {
      "productoid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "dosis": "1 tableta cada 12 horas",
      "duracion": "7 días"
    }
  ],
  "peso": 0,
  "fechaProximaVisita": "2026-05-16"
}
```

Responses:

201: Registro médico creado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/historial/{registroId}: Obtener registro médico por ID

Descripción: Permite consultar la información detallada de un registro médico específico.

Parámetros:

registroId string(\$uuid) (path): Identificador único del registro médico.

Responses:

200: Registro médico.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/historial/{registroId}: Editar registro médico

Descripción: El veterinario puede corregir un registro únicamente durante las primeras 24 horas después de haberlo creado. Pasado ese tiempo el registro queda bloqueado.

Roles permitidos: veterinario, solo el creador del registro.

Parámetros:

registroId string(\$uuid) (path): Identificador único del registro médico.

Ejemplo de request:

```
{
  "citald": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "motivoVisita": "string",
  "diagnostico": "string",
  "tratamiento": "string",
  "medicamentosPrescritos": [
```

```
{
  "productid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "dosis": "1 tableta cada 12 horas",
  "duracion": "7 días"
},
"peso": 0,
"fechaProximaVisita": "2026-05-16"
}
```

Responses:

200: Registro actualizado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: No tiene permiso para editar este registro o han pasado más de 24 horas.

404: El recurso solicitado no existe.

- **Vacunación**

/mascotas/{mascotaid}/vacunas: Consultar vacunas de una mascota

Descripción:

Roles permitidos: veterinario, recepcionista, administrador, y el cliente dueño de la mascota.

Parámetros:

mascotaid string(\$uuid) (path): Identificador único de la mascota.

Responses:

200: Listado de vacunas aplicadas.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/mascotas/{mascotaid}/vacunas: Registrar vacuna aplicada

Descripción: El veterinario registra una vacuna aplicada dentro del historial de la mascota.

Roles permitidos: veterinario.

Parámetros:

mascotaId string(\$uuid) (path): Identificador único de la mascota.

Ejemplo de request:

```
{  
  "nombre": "Parvovirus + Distemper",  
  "productId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",  
  "fechaAplicacion": "2026-05-16",  
  "fechaProximaDosis": "2026-05-16",  
  "observaciones": "string"  
}
```

Responses:

201: Vacuna registrada.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/vacunas/proximas-a-vencer: Listar mascotas con vacunas próximas a vencer

Descripción: Devuelve las mascotas cuyas vacunas vencen en los próximos N días.

Por defecto, el sistema consulta las vacunas próximas a vencer en los siguientes 30 días.

Roles permitidos: veterinario, recepcionista.

Parámetros:

dias integer (query): Días de anticipación para la alerta.

Responses:

200: Mascotas con vacunas próximas a vencer.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

- Hospitalización

/mascotas/{mascotald}/hospitalizacion: Internar mascota

Descripción: Permite registrar la hospitalización de una mascota. Cuando el veterinario interna la mascota, su estado cambia a hospitalizada.

Roles permitidos: Veterinario.

Parámetros: mascotald

Body:

```
{
  "motivoIngreso": "string",
  "veterinarioId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "observacionesIniciales": "string"
}
```

Responses:

201: Mascota internada correctamente.

400: La mascota ya está hospitalizada.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe..

/hospitalizacion/{hospitalizacionId}/evolucion: Registrar nota de evolución diaria

Descripción: Permite registrar una nota de evolución diaria durante el tiempo que una mascota permanezca internada. La nota es registrada por el veterinario responsable y queda asociada a la hospitalización correspondiente.

Roles permitidos: Veterinario.

Parámetros: hospitalizacionId:

Ejemplo Response:

```
{
  "nota": "string"
}
```

Responses:

201: Nota de evolución registrada correctamente.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/hospitalizacion/{hospitalizacionId}/alta: Dar de alta a la mascota

Descripción: Permite registrar el alta de una mascota hospitalizada. Al registrar el alta, el estado de la mascota se actualiza según el estado de egreso:

recuperada → la mascota queda en estado activa.

fallecida → la mascota queda en estado fallecida.

trasladada → la mascota queda en estado activa.

Roles permitidos: Veterinario.

Parámetros: hospitalizacionId

Ejemplo response:

```
{
  "estadoEgreso": "recuperada",
  "observaciones": "string"
}
```

Responses:

200: Alta registrada correctamente.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- Inventario

/inventario: Listar productos del inventario

Descripción:

Roles permitidos: administrador, veterinario, recepcionista.

Parámetros:

tipo string (query): Filtrar por tipo de producto.

Valores permitidos: medicamento, vacuna, insumo_quirurgico.

stockBajo boolean (query): Si es true, devuelve solo los productos por debajo del stock mínimo.

proximoAVencer boolean (query): Si es true, devuelve solo los productos próximos a vencer en los próximos 30 días.

page integer (query): Número de página (base 0).

size integer (query): Elementos por página.

Responses:

200: Lista paginada de productos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/inventario: Registrar producto en inventario

Descripción:

Roles permitidos: administrador.

Parámetros:

No tiene parámetros.

Ejemplo de request:

```
{  
  "nombre": "string",  
  "tipo": "medicamento",  
  "stockDisponible": 0,  
  "stockMinimo": 0,  
  "precio": 0,  
  "fechaVencimiento": "2026-05-16",  
  "descripcion": "string"  
}
```

Responses:

201: Producto registrado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/inventario/{productoid}: Obtener producto por ID

Descripción: Permite consultar la información detallada de un producto específico del inventario.

Parámetros:

productoid string(\$uuid) (path): Identificador único del producto.

Responses:

200: Datos del producto.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/inventario/{productoid}: Actualizar producto del inventario

Descripción: Permite ajustar stock, precio y fecha de vencimiento. Los ajustes manuales de stock quedan registrados en auditoría.

Roles permitidos: administrador.

Parámetros:

productoid string(\$uuid) (path): Identificador único del producto.

Ejemplo de request:

```
{
  "nombre": "string",
  "tipo": "medicamento",
  "stockDisponible": 0,
  "stockMinimo": 0,
  "precio": 0,
  "fechaVencimiento": "2026-05-16",
  "descripcion": "string"
}
```

Responses:

200: Producto actualizado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- Servicios

/servicios: Listar servicios del catálogo

Descripción: Por defecto devuelve solo los servicios activos. Con `incluirInactivos=true` devuelve todos los servicios, incluyendo los inactivos, solo para administrador.

Roles permitidos: todos los roles autenticados.

Parámetros:

incluirInactivos boolean (query): Si es true, incluye servicios inactivos. Valor por defecto: false.

Responses:

200: Lista de servicios.

401: Token de autenticación ausente o inválido.

/servicios: Crear nuevo servicio

Descripción:

Roles permitidos: administrador.

Parámetros:

No tiene parámetros.

Ejemplo de request:

```
{  
  "nombre": "Consulta general",  
  "descripcion": "string",  
  "precio": 0  
}
```

Responses:

201: Servicio creado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/servicios/{servicioid}: Obtener servicio por ID

Descripción: Permite consultar la información detallada de un servicio específico del catálogo.

Parámetros:

servicioid string(\$uuid) (path): Identificador único del servicio.

Responses:

200: Datos del servicio.

401: Token de autenticación ausente o inválido.

404: El recurso solicitado no existe.

/servicios/{servicioid}: Editar servicio del catálogo

Descripción: Permite modificar nombre, descripción y precio.

Roles permitidos: administrador.

Parámetros:

servicioid string(\$uuid) (path): Identificador único del servicio.

Ejemplo de request:

```
{  
  "nombre": "Consulta general",  
  "descripcion": "string",  
  "precio": 0  
}
```

Responses:

200: Servicio actualizado.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/servicios/{servicioid}/desactivar: Desactivar servicio del catálogo

Descripción: Un servicio desactivado no puede seleccionarse en nuevas citas, pero sigue visible en el historial de citas y facturas donde fue usado.

Roles permitidos: administrador.

Parámetros:

servicioid string(\$uuid) (path): Identificador único del servicio.

Responses:

200: Servicio desactivado.

400: El servicio ya está desactivado.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- Facturación

/facturas: Listar facturas

Descripción: Permite consultar una lista paginada de facturas registradas en el sistema. El cliente solo puede consultar sus propias facturas. El recepcionista y el administrador pueden consultar todas las facturas.

Roles permitidos: Cliente, Recepcionista y Administrador.

Parámetros: clientId, estado, fechaDesde, fechaHasta, page, size

Responses:

200: Lista paginada de facturas.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/facturas: Generar factura

Descripción: Permite generar una factura al terminar una atención veterinaria. La factura refleja los servicios prestados, incluyendo los pagos realizados al momento de agendar la cita, y los medicamentos adicionales despachados. También permite aplicar un descuento al valor total.

Roles permitidos: Recepcionista.

Ejemplo Response:

```
{
  "clientId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "medicamentosAdicionales": [
    {
      "productId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "cantidad": 1
    }
  ],
  "descuento": 100
}
```

Responses:

201: Factura generada correctamente.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/facturas/{facturalId}: Obtener factura por ID

Descripción: Permite consultar la información detallada de una factura registrada en el sistema mediante su identificador único.

Roles permitidos: Cliente, Recepcionista y Administrador.

Parámetros: facturalId:

Responses:

200: Datos de la factura.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/facturas/{facturald}/anular: Anular factura

Descripción: Permite marcar una factura como anulada. Para realizar la anulación, se debe registrar el motivo correspondiente. La anulación no elimina el historial médico asociado ni devuelve automáticamente productos al inventario.

Roles permitidos: Recepcionista y Administrador.

Parámetros: facturald

Ejemplo Response:

```
{
  "motivo": "El cliente solicitó cancelación por cambio de planes."
}
```

Responses:

200: Factura anulada correctamente.

400: La factura ya está anulada o pagada.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

/facturas/{facturald}/pdf: Descargar factura en PDF

Descripción: Permite descargar una factura en formato PDF. El cliente puede descargar únicamente sus propias facturas desde su cuenta. El recepcionista y el administrador pueden descargar cualquier factura registrada en el sistema.

Roles permitidos: Cliente, Recepcionista y Administrador.

Parámetros: facturald

Responses:

200: Archivo PDF de la factura.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

404: El recurso solicitado no existe.

- **Reportes**

/reportes/citas: Reporte de citas por período

Método: GET

Descripción: Permite generar y descargar un reporte en formato PDF con el listado de citas registradas dentro de un rango de fechas.

El reporte incluye información como nombre del cliente, mascota, veterinario asignado y estado final de cada cita.

Roles permitidos: Administrador.

Parámetros:

fechaDesde: Fecha de inicio del rango en formato YYYY-MM-DD.

fechaHasta: Fecha de fin del rango en formato YYYY-MM-DD.

Responses:

200: Reporte PDF de citas.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/reportes/facturacion: Reporte de facturación por período

Método: GET

Descripción: Permite generar y descargar un reporte en formato PDF con el resumen de las facturas generadas dentro de un rango de fechas.

El reporte incluye información como total facturado, descuentos aplicados y estado de las facturas.

Roles permitidos: Administrador.

Parámetros:

fechaDesde: Fecha de inicio del rango en formato YYYY-MM-DD.

fechaHasta: Fecha de fin del rango en formato YYYY-MM-DD.

Responses:

200: Reporte PDF de facturación.

400: Datos de entrada inválidos o incompletos.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/reportes/inventario: Reporte de inventario actual

Método: GET

Descripción: Permite generar y descargar un reporte en formato PDF con el estado actual del inventario.

El reporte incluye el listado completo de productos, stock disponible, precio unitario, fecha de vencimiento e identificación clara de productos críticos.

Roles permitidos: Administrador.

Parámetros:

No aplica.

Responses:

200: Reporte PDF de inventario.

401: Token de autenticación ausente o inválido.

403: El rol del usuario no tiene permiso para ejecutar esta acción.

/reportes/historial-acciones: Reporte de historial de acciones

Descripción: Permite generar y descargar un reporte en formato PDF con el historial de acciones registradas en el sistema dentro de un rango de fechas. El reporte incluye información como usuario, rol, acción realizada y fecha/hora exacta del evento. Para generar este reporte, bh-core consulta los datos registrados en bh-audit.

Roles permitidos: Administrador.

Parámetros:

fechaDesde: Fecha de inicio del rango en formato YYYY-MM-DD.

fechaHasta: Fecha de fin del rango en formato YYYY-MM-DD.

usuariold

tipoAccion

Valores permitidos para tipoAccion:

REGISTRO_USUARIO

VERIFICACION_CORREO

APROBACION_CUENTA

RECHAZO_CUENTA

LOGIN_EXITOSO

LOGIN_FALLIDO

CREACION_CITA

CAMBIO_ESTADO_CITA

PAGO_CITA

CREACION_HISTORIAL

EDICION_HISTORIAL

REGISTRO_VACUNA

INICIO_HOSPITALIZACION

ALTA_HOSPITALIZACION

CREACION_FACTURA

ANULACION_FACTURA

AJUSTE_INVENTARIO

CREACION_SERVICIO

EDICION_SERVICIO

DESACTIVACION_SERVICIO
SUSPENSION_USUARIO

Responses:

- 200:** Reporte PDF del historial de acciones.
- 400:** Datos de entrada inválidos o incompletos.
- 401:** Token de autenticación ausente o inválido.
- 403:** El rol del usuario no tiene permiso para ejecutar esta acción.

Schemas definidos en el contrato

Los schemas son estructuras reutilizables que describen los datos que recibe y responde la API. En el microservicio bh-core, estos schemas representan los modelos principales del sistema, como usuarios, clientes, mascotas, citas, historiales médicos, vacunas, hospitalizaciones, productos, servicios y facturas.

Estos schemas permiten definir los campos de cada objeto, sus tipos de datos, valores permitidos, restricciones y propiedades obligatorias. Además, ayudan a mantener el contrato OpenAPI organizado, evitando repetir la misma estructura en diferentes endpoints.

Algunos schemas definidos para bh-core son:

- **RegistroRequest:** estructura usada para registrar un nuevo usuario.
- **LoginRequest:** estructura usada para iniciar sesión.
- **LoginResponse:** estructura de respuesta con el token JWT y los datos del usuario.
- **Usuario:** representa la información de un usuario del sistema.
- **Cliente:** representa los datos de un cliente de la clínica.
- **Mascota:** representa los datos de una mascota asociada a un cliente.
- **Cita:** representa una cita agendada en el sistema.
- **CitaRequest:** estructura usada para crear una nueva cita.
- **PagoRequest:** estructura usada para registrar el pago al agendar una cita.
- **HistorialMedico:** representa un registro clínico de una mascota.
- **HistorialMedicoRequest:** estructura usada para registrar o editar una consulta médica.
- **Vacuna:** representa una vacuna aplicada a una mascota.
- **VacunaRequest:** estructura usada para registrar una vacuna aplicada.
- **Hospitalizacion:** representa el proceso de internamiento de una mascota.
- **Producto:** representa un medicamento, vacuna o insumo del inventario.
- **ProductoRequest:** estructura usada para crear o actualizar productos del inventario.
- **Servicio:** representa un servicio ofrecido por la clínica.

- **ServicioRequest:** estructura usada para crear o editar servicios.
- **Factura:** representa una factura generada en el sistema.
- **FacturaRequest:** estructura usada para generar una factura.
- **ErrorResponse:** estructura estándar para respuestas de error.
- **PaginationMeta:** información de paginación usada en listados.

Schemas	
RolUsuario > [...]	↩
EstadoCuenta > [...]	↩
EstadoMascota >	↩
EstadoCita >	↩
EstadoFactura >	↩
TipoProducto >	↩
EstadoEgreso >	↩
TipoAccion >	↩
RegistroRequest >	↩
VerificacionCorreoRequest >	↩
LoginRequest >	↩
MotivoRequest >	↩

ClienteRequest >	↩
MascotaRequest >	↩
CitaRequest >	↩
PagoRequest >	↩
HistorialMedicoRequest >	↩
MedicamentoPrescritoRequest >	↩
VacunaRequest >	↩
HospitalizacionRequest >	↩
NotaEvolucionRequest >	↩
AltaRequest >	↩
ProductoRequest >	↩
ServicioRequest >	↩
FacturaRequest >	↩
ItemFacturaRequest >	↩

MensajeResponse >	↩
ErrorResponse >	↩
LoginResponse >	↩
Usuario >	↩
Ciente >	↩
Mascota >	↩
Cita >	↩
Pago >	↩
HistorialMedico >	↩
MedicamentoPrescrito >	↩
Vacuna >	↩
AlertaVacuna >	↩
Hospitalizacion >	↩
NotaEvolucion >	↩
Producto >	↩

Servicio >	↩
Factura >	↩
ItemFactura >	↩
PagedUsuarios >	↩
PagedClientes >	↩
PagedCitas >	↩
PagedHistorialMedico >	↩
PagedProductos >	↩
PagedFacturas >	↩
PaginationMeta >	↩

Conclusión

Se diseñó y documentó el contrato OpenAPI del microservicio bh-core siguiendo el enfoque API First. El documento evidencia los endpoints principales del sistema, incluyendo métodos HTTP, rutas, parámetros, cuerpos de solicitud, respuestas exitosas y códigos de error.

La especificación fue construida en Swagger Editor y servirá como base para la implementación del microservicio bh-core, garantizando que el equipo desarrolle sobre un contrato previamente definido y validado.